Comparison of The Floyd-Warshall-Roy and Dijkstra's Algorithms

This paper will investigate the conjecture below.

```
Even for relatively small values of k, the Floyd-Warshall-Roy algorithm will
provide better performance in practice for the k-source minimum cost path
problem than k runs of Dijkstra's algorithm (regardless of the asymptotic
running times).
```

Introduction

The minimum cost path (MCP) problem is a vital concept in various fields, ranging from packet travel analysis in computer networking to spatial modeling in Geographic Information Systems (GIS). In GIS applications, for example, MCP algorithms are used to simulate the strategic movements of ancient civilizations by optimizing traversal costs based on variables like seclusion, access to farmland, defensibility, and viewshed to test theories of human psychology [1]. A specific extension of this is the k-source minimum cost path (KMCP) problem, which focuses on finding the optimal routes from a designated subset of k source vertices to all other vertices within the graph.

This paper will define the KMCP problem involves being given a directed weighted graph with no negative edges, and a set S of k source vertices, the goal of the solution algorithm implementations will be to construct a minimum-cost path tree for each source in S.

There are two methods of developing solutions to this problem, an all-pairs algorithm such as the Floyd-Warshall-Roy algorithm can be used to find all of the minimum cost path trees for every possible starting vertex, and return those corresponding to the starting vertices in S. Alternatively, we could run Dijkstra's algorithm on each starting vertex in S.

Algorithms in Theory

In the remainder of this paper, any use of the term Dijkstra's algorithm refers specifically to the binary min-heap implementation. This implementation is used because the conjecture concerns practical performance, and the heap-based version is the more meaningful practical baseline. Comparing Floyd-Warshall-Roy against a less efficient linear-search implementation would not test the conjecture as strongly. While other algorithms exist for minimum cost paths, they fall outside the scope of this comparison. Bellman-Ford, for example, is unnecessary because the test graphs contain no negative edge weights.

This paper will define space complexity as the peak amount of space that each algorithm takes up during the execution of the algorithm. This will not include the inputs or the outputs.

The Floyd-Warshall-Roy algorithm solves the problem by computing the minimum cost between every ordered pair of vertices. For a graph G, with vertex set V, its asymptotic running time is $O\left(|V|^3\right)$. Because it computes all-pairs information, its main memory usage is the storage of a |V| row by |V| column matrix storing costs of travel, giving a asymptotic space complexity of $O\left(|V|^2\right)$.

For the k-source minimum cost path problems, Dijkstra's algorithm can instead be run one for each source vertex in S. Using a binary min-heap implementation, one run takes $O\left((|V| + |E|)\log_2|V|\right)$, thus k runs will take $O\left(k\left(|V| + |E|\right)\log_2|V|\right)$. Its space complexity is $O\left(|V|\right)$.

The asymptotic runtimes and space complexities provide theoretical background, but they are not direct predictors of a practical runtime. Therefore, this paper uses asymptotic analysis only to motivate qualitative expectations, and evaluates practical performance using runtime measurements from baseline C++ implementations.

Methodology

Dominant operation count model

This study will evaluate the performance of the Floyd-Warshall-Roy algorithm and Dijkstra's algorithm using two approaches: (i) theoretical expectations for their runtime, and (ii) empirical measurements from baseline C++ implementations.

This paper uses a model based on a simplified count of the asymptotically dominant algorithmic work units for each algorithm. For Floyd-Warshall-Roy, this is the number of distance-improvement checks over all (i,j,k) triples. For heap-based Dijkstra, this is the number of extract-min and edge-relaxation/update operations including the heap maintenance steps. For this paper, this count model will be referred to as the the dominant operation count.

While these counts model the algorithmic scaling, they are theoretical approximations rather than exact runtime predictors. A single dominant operation in Dijkstra may take longer to execute than one in FWR due to complex memory access patterns and heap overhead. Therefore, this operation model will establish theoretical scaling, while empirical C++ measurements will determine true practical performance.

Implementations

As far as runtime is concerned, each of the two algorithms shall act as a black box with the exact same functionality defined as:

Input:
- A directed and weighted graph G with no negative edges
- A set S = {$s_1, s_2, \ldots, s_k$} of starting vertices
Output:
- A minimum-cost path tree for each starting vertex in S

Implementation and environment.
Both the baseline solution algorithm variants are implemented in C++20 and compiled using GNU g++ and adapt B. Bird's pseudocode [2]/[3]

Experiments are executed on an AMD Ryzen 7 7840HS processor with 16GB of installed physical memory (RAM) running Ubuntu on Windows 11. All experiments use a fixed random seed for reproducibility.

The creation of the test files that the algorithm variants use was done using python 3, primarily due to ease of use.

The Floyd-Warshall-Roy implementation initializes a cost matrix and a parent matrix of size $|V| \times |V|$, sets each diagonal entry to 0, loads each directed edge weight into the initial cost matrix, and then performs the standard triple nested relaxation over all ordered triples (i, j, k). After all relaxations are complete, the implementation returns only the minimum-cost path trees corresponding to the source vertices in S.

The heap-based Dijkstra implementation executes once for each source vertex in S. Each run stores, for every vertex, the current best known minimum cost from the source and its parent. A binary min-heap was used to repeatedly select the next vertex of minimum cost. Whenever a cheaper path to a neighboring vertex was found, that vertex was inserted into the heap with its updated key, and stale heap entries were ignored when removed. In the remainder of this investigation, any reference to Dijkstra's algorithm refers specifically to this binary min-heap implementation.

Generating tests and data
A Python program (`make_test.py`) uses a seeded random number generator (seed = 1) to produce reproducible test files containing random directed weighted graphs. These tests are parameterized by vertex count $|V|$, graph density d, and source count k. The generation follows two primary steps:

- **Edge generation:** For a given $|V|$ and d, the target edge count is calculated as $|E| = d|V|(|V| - 1)$. Ordered vertex pairs (u, v) with $u \neq v$ are repeatedly sampled until the required number of distinct directed edges is constructed.
- **Weight assignment:** Edge weights are sampled uniformly from the interval $[1, 100]$ to guarantee no negative edges, satisfying the strict requirements of Dijkstra's algorithm.

For the $|V| = 150$ trials, generated tests utilized densities of $d \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ and source counts of $k \in \{10, 20, 30, 40, 50, 70, 90, 110, 130, 150\}$. This generator structure functions identically for other requested values of $|V|$ and k.

Test Execution and Reporting
The test file produced by (`make_test.py`) is parsed through a C++ runner program (`runner.cpp`), which reconstructs the graph and applies both algorithms to the exact same input instance. A sample test file utilizes the following command structure:

```
- N (Defines the total number of vertices in the graph)
- E (Defines a directed edge with assigned weight)
- S (Defines the source vertexes to process)
- RUN (Triggers the execution of both algorithms)
```

To ensure precise profiling, the runner measures the wall-clock runtime of both Floyd-Warshall-Roy and the heap-based Dijkstra using the C++ `chrono` library.

Each test case is repeated a reasonable number of times for the given $|V|$ to record the average runtime in milliseconds. Furthermore, the runner validates correctness by making sure that both implementations output identical minimum-cost values for every source-destination pair before any runtime comparisons are analyzed.

Theoretical Prediction

For a directed and edge-weighted graph G with no negative cost edges, this paper will define graph density to be the ratio of edges in G over the maximum possible edges $|V|\,(|V|-1)$.

$$density = \frac{|E|}{|V|\,(|V|-1)}$$

The Floyd-Warshall-Roy algorithm performs one distance improvement check for each (i, j, k) combination, so its dominant operation count is modeled by:

$$O_{FWR}\,(|V|) = |V|^3$$

For k runs of heap-based Dijkstra's algorithm, we model the dominant operations by combining the $k\,|E|$ edge-relaxation checks with the $\log_2 |V|$ heap operations required:

$$O_D\,(|V|, |E|, k) = k\,(|E| + |V|) \log_2 |V|$$

Data and hypothesis

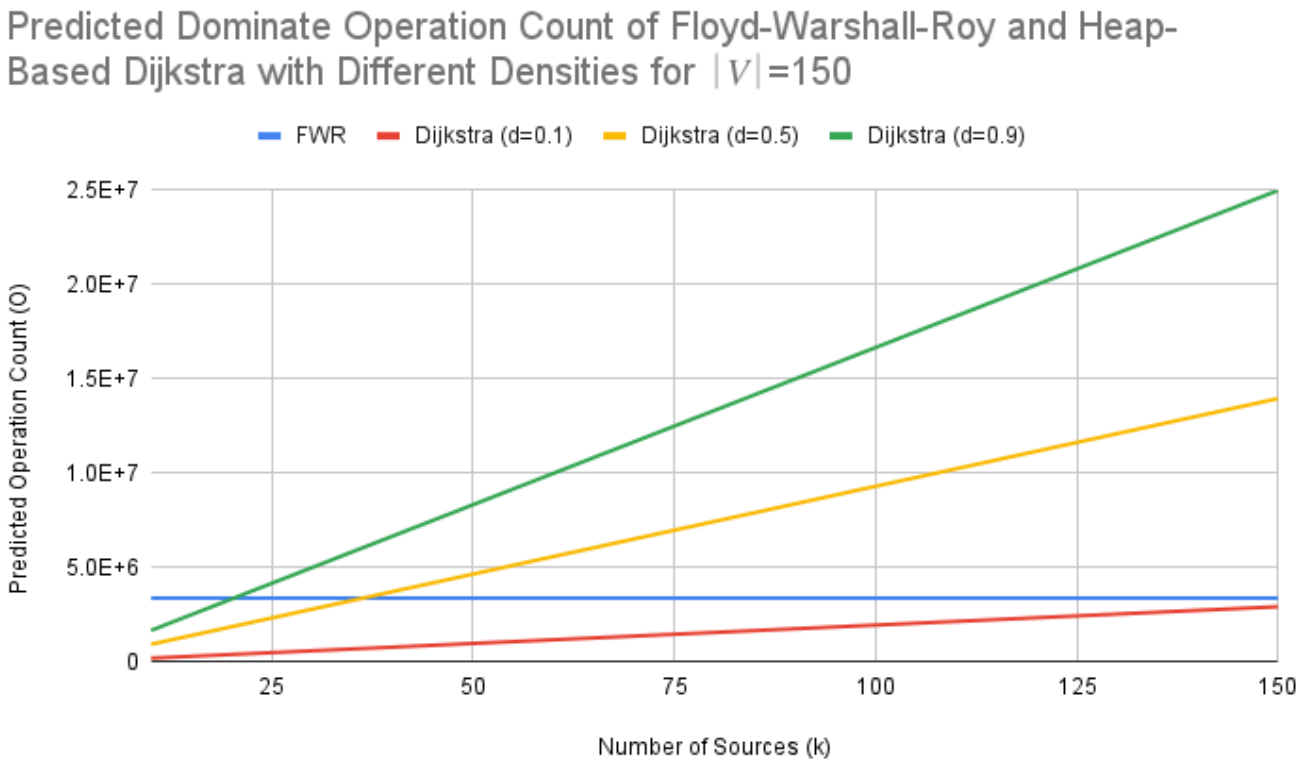

Figure 1. Predicted dominate operation count of Floyd-Warshall-Roy and heap-based Dijkstra for $|V| = 150$ as k and graph densities vary.

The above graph allows us to see that as the number of sources increases, the dominate operation count of each implementation of Dijkstra, regardless of density, grows while Floyd-Warshall-Roy stays constant. We can also see that sparse Dijkstra with a density of 0.1 doesn't cross Floyd-Warshall-Roy in dominate operation counts (at least for valid values of $k \leq |V|$). We can find the cross-over density at which Dijkstra requires less dominate operation's than Floyd-Warshall-Roy under $k = |V|$ for a vertex count of 150

$$|V|^3 = k\left(|E| + |V|\right) \cdot \log_2 |V|$$

$$150^3 = |E|150\log_2\left(150\right) + 150^2\log_2\left(150\right)$$

$$|E| = 2962.54$$

$$density\left(d\right) = \frac{|E|}{|V|\left(|V| - 1\right)}$$

$$d = \frac{2962.54}{150\left(149\right)}$$

$$d = 0.1326$$

If we predict that this will hold for all valid |V|, we can make the following hypothesis:

[H.1] There exists a calculable crossover density d such that, for a graph with fixed $|V|$ and a source count k, decreasing the graph density lower past this value will result in Dijkstra's algorithm being more efficient for any $0 < k \leq |V|$.

Building on this idea, we can model the crossover source count k as a function of the density d.

$$|V|^3 = k\left(|E| + |V|\right) \cdot \log_2 |V|$$

Suppose

$$|E| = d\,|V|\left(|V| - 1\right)$$

Thus we have that

$$|V|^3 = k\left(d\,|V|\left(|V| - 1\right) + |V|\right)\log_2 |V|$$

$$\frac{|V|^3}{\left(d\,|V|\left(|V| - 1\right) + |V|\right)\log_2 |V|} = k$$

Letting |V|=150 we get the following graph

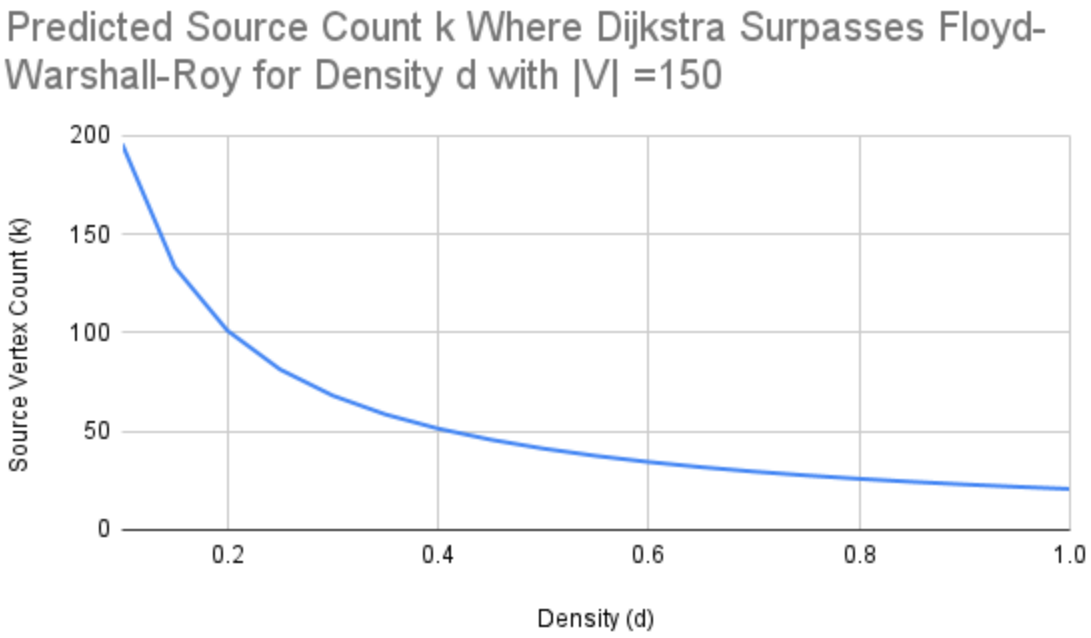

Figure 2. Predicted crossover source count k, for Floyd-Warshall-Roy and Heap-based Dijkstra when |V| = 150, Each point represents the value of
k at which the two work models are equal for a given density.

From figure 2, we can predict that as density (d) increases, the source vertex count (k) required for Floyd-Warshall-Roy to be more performant than Dijkstra goes down. Predicting that this is true for all values of |V|, we make the following hypothesis:

[H.2] For fixed graph size, increasing graph density is expected to decrease the source count k at which Floyd–Warshall–Roy becomes as fast as or faster than heap-based Dijkstra.

Results

Data charts

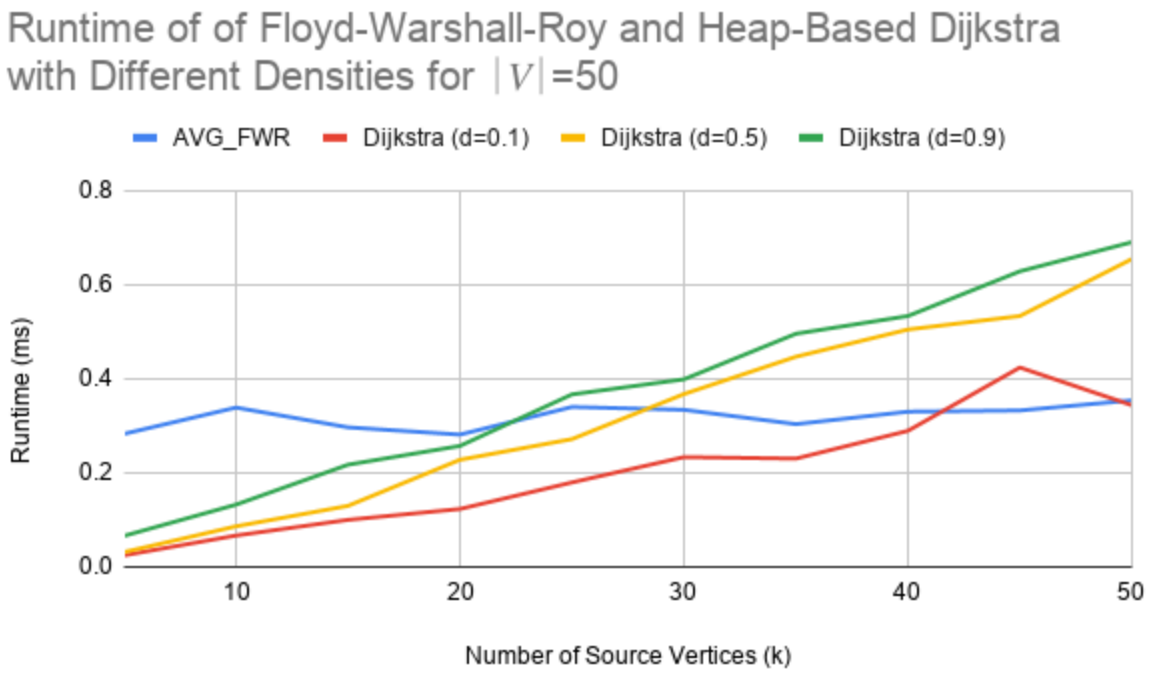

Figure 3. Runtime of Floyd-Warshall-Roy and Heap-Based Dijkstra with different densities for $|V|$=50, with each time measurement repeated 25 times.

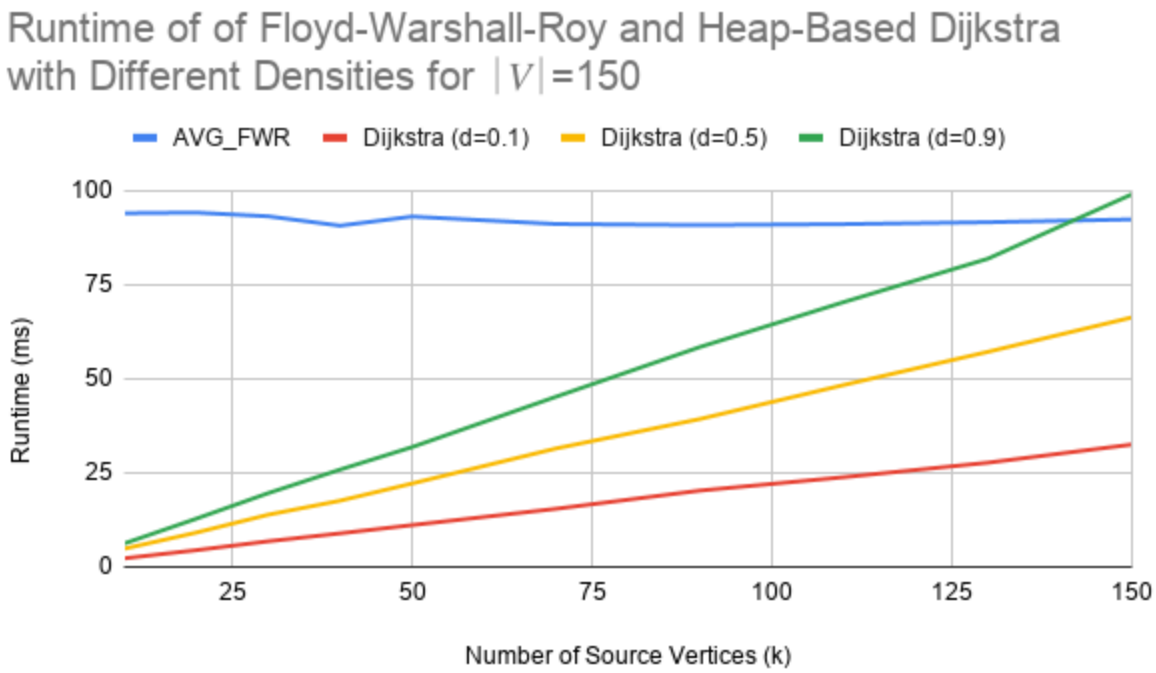

Figure 4. Runtime of Floyd-Warshall-Roy and Heap-Based Dijkstra with different densities for $|V|$=150, with each time measurement repeated 25 times.

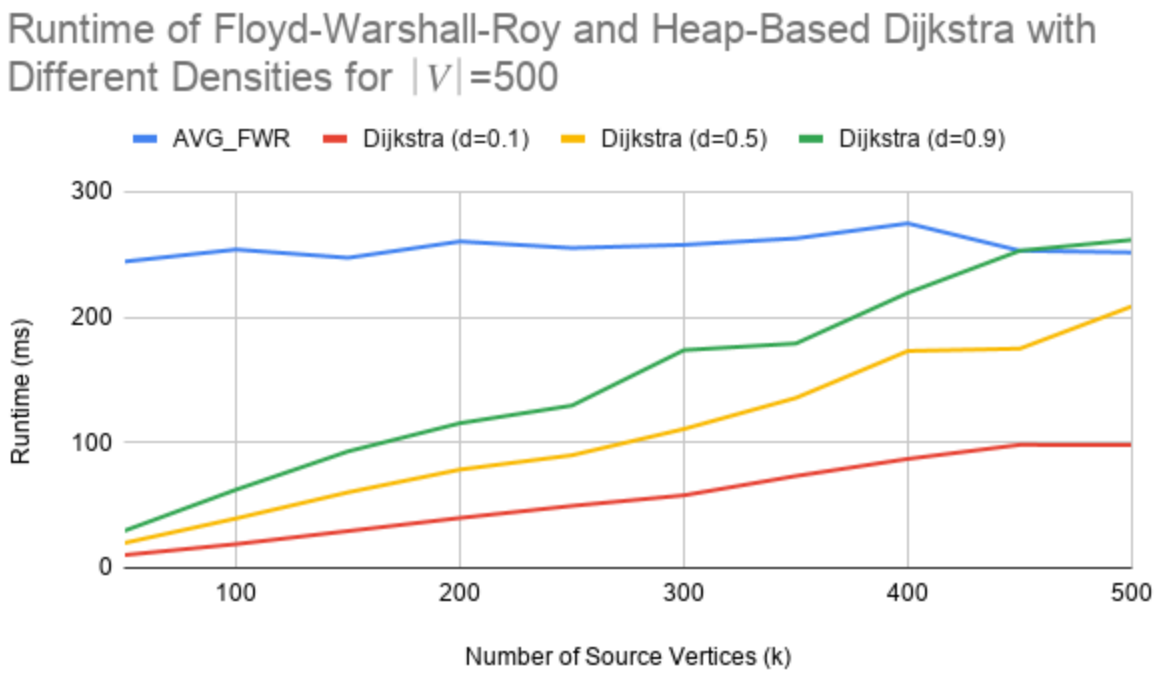

Figure 5. Runtime of Floyd-Warshall-Roy and Heap-Based Dijkstra with different densities for $|V|$=500, with each time measurement repeated 3 times.

|V|	AVG_FWR	Dijkstra d=0.1 (ms)	Dijkstra d=0.5 (ms)	Dijkstra d=0.9 (ms)
50	0.35	0.34	0.65	0.69
150	92.33	32.51	66.29	99.006
500	251.46	97.95	208.46	261.47

Figure 6. Table of observed runtime for $k = |V|$ Floyd-Warshall-Roy and Dijkstra at three different densities (0.1, 0.5, 0.9).

Testing [H.1]

The data provided in figure 6 confirms that such a crossover density exists, where below a certain density, Dijkstra is more performant for any valid k. At a graph density of $d = 0.1$, Dijkstra consistently outperforms FWR at $k = |V|$ across all tested graph sizes:

- For $|V| = 50$: Dijkstra (0.34 ms) < FWR (0.35 ms)
- For $|V| = 150$: Dijkstra (32.51 ms) < FWR (92.34 ms)
- For $|V| = 500$: Dijkstra (97.95 ms) < FWR (251.46 ms)

To find the precise crossover density (d_{cross}) where the two algorithms have equal runtimes, we apply *linear interpolation* between the recorded density intervals (d_1 and d_2):

$$d_{cross} = d_1 + (d_2 - d_1)\frac{T_{FWR} - T_{D1}}{T_{D2} - T_{D1}}$$

Using the table data, the calculated empirical crossover densities are:

- $|V| = 50$: Interpolating between $d = 0.1$ and $d = 0.5$ yields $d_{cross} \approx 0.113$.
- $|V| = 150$: Interpolating between $d = 0.5$ and $d = 0.9$ yields $d_{cross} \approx 0.818$.
- $|V| = 500$: Interpolating between $d = 0.5$ and $d = 0.9$ yields $d_{cross} \approx 0.824$.

These findings successfully validate [H.1], showing that Dijkstra maintains a experimental runtime advantage over FWR up to these specific density thresholds.

Testing [H.2]
In Figures 3, 4, and 5 we can see the crossover points where the Floyd-Warshall-Roy algorithm becomes more efficient than Dijkstra's. As graph density increases, the threshold for the number of source vertices (k) required for Dijkstra's to exceed the Floyd-Warshall-Roy runtime also decreases.

Figure 1 provides the clearest example of this relationship. Using the linear interpolation formula, we can approximate the specific k values where Dijkstra's implementations at various densities surpass the Floyd-Warshall-Roy runtime:

$$k = k_1 + (k_2 - k_1)\frac{F_1 - D_1}{(D_2 - D_1) - (F_2 - F_1)}$$

Plugging in the values from the chosen rows k=10, and k=45

k	AVG_FWR	Dijkstra d=0.1 (ms)	Dijkstra d=0.5 (ms)	Dijkstra d=0.9 (ms)
10	0.339	0.067	0.086	0.132
45	0.332	0.424	0.533	0.628

We get the crossover values of k

	Dijkstra (d=0.1)	Dijkstra (d=0.5)	Dijkstra (d=0.9)
k	36.201	29.494	24.38155881

Showing that as density increases, the k value for where the runtime of Floyd-Warshall-Roy has a better runtime decreases, thus proving [H.2]

Conclusions and limitations

Conclusion
This paper investigated the conjecture that even for relatively small values of k, the Floyd-

Warshall-Roy algorithm will provide better practical performance for the k-source minimum cost path problem than k runs of heap-based Dijkstra's algorithm. Based on the theoretical prediction and the runtime measurements, the Floyd-Warshall-Roy algorithm maintains a constant runtime as the number of source vertices increases, while the runtime of Dijkstra's algorithm scales linearly. This difference creates a crossover point where the all-pairs algorithm eventually becomes more efficient than multiple runs of the single-source algorithm.

The data shows that this crossover is highly dependent on graph density. In sparse graphs, Dijkstra's algorithm remains more performant even for high values of k. As density increases, the source count required for Floyd-Warshall-Roy to be faster decreases. Therefore, while the conjecture is true for very dense graphs with high source counts, Dijkstra's algorithm is the more efficient choice for most practical k-source minimum cost path problems.

Limitations
Several limitations must be acknowledged to contextualize the results.

- Ignoring experimental memory performance: While runtime is a large part of performance, managing memory can also be very important for devices that have less, or for when the graphs are dealing with massive details.
- Limited test cases: The investigation only used a limited set of graph densities, graph sizes, and source counts. Because of this, the exact crossover points are only approximations based on the tested values and interpolation.
- Random graph generation: The test data was produced using seeded random directed weighted graphs. This made the results reproducible, but these graphs do not represent every possible graph structure that could appear in practice.
- One implementation choice: This paper only compared the standard Floyd-Warshall-Roy implementation against heap-based Dijkstra using a binary min-heap. Different implementation choices could change the practical runtime results such as Fibonacci Heaps, or Dijkstra implementations with linear scanning.
- One Environment: These results are based on baseline C++ implementations and specific hardware. Performance may vary on different processors or with different compiler optimizations.
- Graph Types: The investigation only utilized random directed weighted graphs with no negative edges. Results might differ for specific graph structures, such as graphs with negative edge weights (where Dijkstra is not applicable).

Bibliography

[1] G. Davies, "GIS approaches to understanding connections and movement through space," Ph.D. dissertation, Lancaster Environ. Centre, Lancaster Univ., Lancaster, U.K., 2019

[2] B. Bird, "Algorithm Science (Summer 2025) - 35 - Minimum Cost Paths II," YouTube, 2025. [Online]. Available: https://www.youtube.com/watch?v=ANZpc0zhc9Q&list=PLU4IQLU9e_Orrc7r8qcgbegp03BfMD-rq&index=35

[3] B. Bird, "Algorithm Science (Summer 2025) - 36 - Minimum Cost Paths III," YouTube, 2025. [Online]. Available: https://www.youtube.com/watch?v=BejY7uiUhEs&list=PLU4IQLU9e_Orrc7r8qcgbegp03BfMD-rq&index=36